



GEM_GXL Ethernet Controller Demo Testbench

User Guide

Revision 1.0

CADENCE CONFIDENTIAL

Confidentiality Notice

© 1996-2017 Cadence Design Systems, Inc. All rights reserved.

Cadence Design Systems, Inc. (Cadence), 2655 Seely Ave., San Jose, CA 95134, USA.

Trademarks: Trademarks and service marks of Cadence Design Systems, Inc. contained in this document are attributed to Cadence with the appropriate symbol. For queries regarding Cadence's trademarks, contact the corporate legal department at the address shown above or call 800.862.4522. All other trademarks are the property of their respective holders.

Restricted Permission: This document is protected by copyright law and international treaties and contains trade secrets and proprietary information owned by Cadence. Unauthorized reproduction or distribution of this document, or any portion of it, may result in civil and criminal penalties. Except as specified in this permission statement, this document may not be copied, reproduced, modified, published, uploaded, posted, transmitted, or distributed in any way, without prior written permission from Cadence. This document contains the proprietary and confidential information of Cadence or its licensors, and is supplied subject to, and may be used only in accordance with, a written agreement between Cadence and its customer.

Unless otherwise agreed to by Cadence in writing, this statement grants Cadence customers permission to print one (1) hard copy of this document subject to the following conditions:

1. This document may not be modified in any way.
2. Any authorized copy of this document or portion thereof must include all original copyright, trademark, and other proprietary notices and this permission statement.
3. The information contained in this document cannot be used in the development of like products or software, whether for internal or external use, and shall not be used for the benefit of any other party, whether or not for consideration.

Disclaimer: INFORMATION IN THIS DOCUMENT IS SUBJECT TO CHANGE WITHOUT NOTICE AND DOES NOT REPRESENT A COMMITMENT ON THE PART OF CADENCE. EXCEPT AS MAY BE EXPLICITLY SET FORTH IN A WRITTEN AGREEMENT BETWEEN CADENCE AND ITS CUSTOMER, CADENCE DOES NOT MAKE, AND EXPRESSLY DISCLAIMS, ANY REPRESENTATIONS OR WARRANTIES AS TO THE COMPLETENESS, ACCURACY OR USEFULNESS OF THE INFORMATION CONTAINED IN THIS DOCUMENT. CADENCE DOES NOT WARRANT THAT USE OF SUCH INFORMATION WILL NOT INFRINGE ANY THIRD PARTY RIGHTS, AND CADENCE DISCLAIMS ALL IMPLIED WARRANTIES, INCLUDING MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE. CADENCE DOES NOT ASSUME ANY LIABILITY FOR DAMAGES OR COSTS OF ANY KIND THAT MAY RESULT FROM USE OF SUCH INFORMATION. CADENCE CUSTOMER HAS COMPLETE CONTROL AND FINAL DECISION-MAKING AUTHORITY OVER ALL ASPECTS OF THE DEVELOPMENT, MANUFACTURE, SALE AND USE OF CUSTOMER'S PRODUCT, INCLUDING, BUT NOT LIMITED TO, ALL DECISIONS WITH REGARD TO DESIGN, PRODUCTION, TESTING, ASSEMBLY, QUALIFICATION, CERTIFICATION, INTEGRATION OF CADENCE PRODUCTS, INSTRUCTIONS FOR USE, LABELING AND DISTRIBUTION, AND CADENCE EXPRESSLY DISAVOWS ANY RESPONSIBILITY WITH REGARD TO ANY SUCH DECISIONS REGARDING CUSTOMER'S PRODUCT.

Restricted Rights: Use, duplication, or disclosure by the Government is subject to restrictions as set forth in FAR52.227-14 and DFAR252.227-7013 et seq. or its successor.

Document Revision History

Revision	Date	Changes made in document from previous revision
Rev 0.1	07/03/2017	Initial version.
Rev 0.2	12/04/2017	Various items: - Updated after feedback from verification lead. - The SoC Integration section has been rewritten based on the current progress on the driver tests front. - Better aligned to the Demo Testbench user guide template.
Rev 0.3	28/05/2017	Generally updated to reflect changes in the user guide template: - Updated “Replacing the System Side Busses with Alternative UVCs” section (added more info from the template). - Updated “Installation” section (removed env variables since they are set by the Makefile). - Updated “Directory Structure” section since some changes have been made. - Updated “Running a Simulation” section (removed run_demo.csh which has been descope, plus Makefile updates). - Updated “Running Regressions” section (removed start_vm.sh which has been descope, renamed .vsfis plus Makefile updates). - Updated the test names in Section 4 since they have been changed due to a new requirement.
Rev 0.4	29/05/2017	Updated after feedback from Architect and refined the vManager section.
Rev 0.5	14/06/2017	Various: - Added description of standalone regression summary files. - Added some detail on tests configuration dependency in the known issue section. - Removed references to internal FV flows. - Correctly specified the location of UVM_REG models. - Stated clearly in the directory structure section where the DUT instance is. - Added the QSG to the references and updated citations in the text.
Rev 0.6	19/06/2017	Updates after feedback from demo TB initiative: - Changed path to HTML documentation. - Shortened captions. - Fixed vManager command. - Updated the RAM integration section. - Modified section 2.2 to point out that C reuse is still not available.
Rev 0.7	20/06/2017	Added warning documentation.
Rev 0.8	22/06/2017	Added blanket statements on UVM_REG models.
Rev 1.0	07/11/2017	Updated after environment improvements.

Table of Contents

List of Figures	5
List of Tables	6
List of Acronyms	7
1. Introduction	8
1.1. Simulator and VIPCAT Compatibility	8
1.2. References.....	8
2. Architectural Overview	9
2.1. Common UVM Environment	9
2.2. UVM Test Layer.....	10
2.3. C Test Layer	10
2.4. Cadence GEM_GXL IP	11
3. Package and Directory Structure	13
3.1. Installation.....	13
3.2. Directory Structure	14
3.3. Running a Simulation on a Standalone Package.....	15
3.4. Test Completion Status – Pass/Fail	16
3.5. Running Regressions	17
3.5.1. Types of Regression	17
3.5.2. Using eManager/vManager for regressions	17
3.5.3. Running Standalone Regressions	18
3.6. RAM Integration Tests	19
3.6.1. Current RAM Integration Limitations	19
4. Use Cases/Demo Cases	20
4.1. NaturalDocs UVM-SV VE and Demo Test Documentation.....	22
4.2. Doxygen C Demo Test Documentation.....	23
5. Known Issues	24
5.1. VIP Related Issues	24
5.2. Demo Testbench Related Issues.....	24
5.3. Test or Regression Related Issues	24
Appendix A. Customer Modifications	26
A.1 Replacing the System Side Busses with Alternative UVCs	26
A.1.1 Recommended Tests for Validating System Bus Replacement	27

List of Figures

Figure 1: Demo Testbench Layers	9
Figure 2: C Test Example Structure	11
Figure 3: The GEM_GXL Demo Testbench structure.	11
Figure 4: NaturalDocs documentation example.	22
Figure 5: Doxygen documentation example.	23
Figure 6: Required Customisations System Bus Replacement.	27

List of Tables

Table 1: Simulator and VIPCAT Compatibility.	8
Table 2: Required commands to install the GEM_GXL Demo Testbench.	13
Table 3: Standalone Package Required Environment Variables.	13
Table 4: The directory structure of the GEM_GXL Demo Testbench.	14
Table 5: Available C tests in the GEM_GXL Demo Testbench.	20
Table 6: Available UVM tests in the GEM_GXL Demo Testbench.	20

List of Acronyms

Acronym	Meaning
AMBA	Advanced Microcontroller Bus Architecture
APB	AMBA Peripheral Bus
API	Application Programming Interface
AXI	AMBA eXtensible Interface
DIP	Design IP
DPI	Direct Programming Interface
DUT	Design Under Test
GEM	Gigabit Ethernet MAC
IP	Intellectual Property
LSF	Loading Share Facility
RAM	Random Access Memory
RTL	Register Transfer Level
SoC	System on Chip
UVC	Universal Verification Component
UVM	Universal Verification Methodology
VIP	Verification IP
VIPCAT	VIP Catalogue

1. Introduction

This document defines the Cadence GEM_GXL IP Demo Testbench which has been developed to enable the customer to run simple test cases that demonstrate the Cadence GEM_GXL IP base features. These test cases are abstracted to promote easier understanding of how to program the Cadence GEM_GXL IP and allow reuse within a higher level verification environment. This testbench can also be used as a starting point for the customer to integrate the Cadence VIP relating to the Cadence IP into their system verification environment.

Please note that the Cadence VIP Catalogue (VIPCAT) is an essential component of this Demo Testbench and must be installed and available on the customer's systems.

This section gives a generic overview and displays tool requirements for running the GEM_GXL Demo Testbench. In addition:

- Section 2 (Architectural Overview) details the architecture of the GEM_GXL Demo Testbench.
- Section 3 (Package and Directory Structure) gives information on how installing and running the GEM_GXL Demo Testbench. The directory structure of the environment is also highlighted and notable files location discussed.
- Section 4 (Use Cases/Demo Cases) introduces available test cases and related code documentation which can be extracted using NaturalDocs and Doxygen tools.
- Section 5 (Known Issues) reports for recognized issues inside the GEM_GXL Demo Testbench.

1.1. Simulator and VIPCAT Compatibility

The Cadence GEM_GXL IP Demo Testbench has been tested on the following simulators and VIPCAT versions:

Table 1: Simulator and VIPCAT Compatibility.

Simulator	VIPCAT Version
Cadence Incisive 15.2 (15.20.020)	Cadence VIPCAT 11.3 (11.30.046)

1.2. References

Industry standards

- [1] **IEEE.** IEEE Standard 802.3 for Ethernet. 2015.
- [2] **Accellera.** UVM 1.1 User Guide. 2012. Available online:
http://accellera.org/images/downloads/standards/uvm/uvm_users_guide_1.1.pdf

Cadence documentation

- [3] **Cadence.** GEM_GXL Controller IP, User Guide. 2017.
- [4] **Cadence.** Cadence IPs for 2.5G and Gigabit Ethernet MAC, Quick Start Guide. 2017
- [5] **Cadence.** EMAC FW Driver User Guide. 2016.
- [6] **Cadence.** SoC Integration Using the GEM_GXL Ethernet Controller Demo Testbench AN. 2017

Other documents

- [7] **G. Valure.** NaturalDocs Documentation. Available online:
<http://www.naturaldocs.org/documenting.html>.
- [8] **D. v. Heesch.** Doxygen Manual. Available online:
<http://www.stack.nl/~dimitri/doxygen/manual/index.html>.

2. Architectural Overview

This section describes the Cadence GEM_GXL IP Demo Testbench common architecture which may be shared between variants of the Cadence GEM_GXL IP. Where multiple variants exist, there is a subsection for each IP type.

The Demo Testbench may be delivered with either UVM SV tests or C tests as illustrated by the right-hand columns of Figure 1. Offering these options promotes reuse of the tests in a system context. Both options share a common UVM environment. The UVM test layers or C test layers interface to the common UVM environment through abstraction layers, which in the case of the C test layer also includes C-DPI¹ calls to transition over the language barrier. Figure 1 also illustrates reuse in a CPU-based SoC verification environment on the left-hand column of the diagram (see [6]).

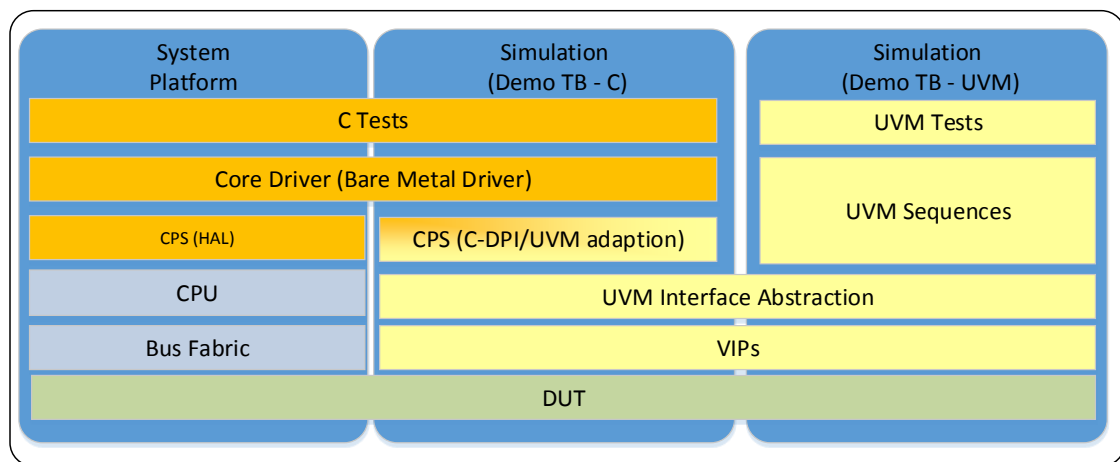


Figure 1: Demo Testbench Layers

2.1. Common UVM Environment

The Demo Testbench is configured and customized depending on the provided type of Cadence GEM_GXL IP, however there are some constant factors:

1. The Demo Testbench uses the UVM class libraries and associated methodology. The Demo Testbench is encapsulated as a UVC called `cdn_gem_demo` and can be found in the following directory:


```
<DEMO_TB_INSTALL_PATH>/verif/uvc_lib/cdn_gem_demo
```
2. The Cadence VIP is used for System bus interfaces (e.g. AXI, APB) and protocol interfaces (e.g. ENET). The VIP is wrapped up as UVCs to make them UVM compatible. These UVCs can be found in the `<DEMO_TB_INSTALL_PATH>/verif/uvc_lib` directory and are named `cdn_*_vip`; where `*` is the name of the protocol (e.g. `enet/axi/apb`).
 - a. NaturalDocs HTML documentation is available for these VIP UVC wrappers. Please refer to Section 4 for more information.
 - b. The Cadence VIP uses the SV interface and SV config object approach instead of the SOMA and Verilog module because it works better with UVM.

¹ The SystemVerilog Direct Programming Interface (DPI) is a lightweight interface designed to map SystemVerilog tasks and functions to foreign language implementation (as C) and vice versa.

- i. For Cadence ENET VIP, all of the required SV interfaces were generated and are located in `cdn_enet_vip/sv`
 - ii. All of the Cadence ENET VIP timing values that need to be configured to match the design IP can be found inside `cdn_enet_demo/sv`.
 - iii. All SV config objects and can be found in the normal `sv` dirs.
- c. Note that these VIP UVC wrappers all inherit from the VIP UVC wrappers that are located inside the `$DENALI/ddvapi/sv` directory and subdirectories.
- 3. The register model is created using the `UVM_REG` classes based on the UVM class library. This model is provided in this testbench. Such a model ensures that register sequences can be written at this level of abstraction and these sequences can be reused by the customer if they also use the provided `UVM_REG` model. If the Demo Testbench is part of a standalone delivery, the `UVM_REG` model can be found as `cdn_gem_demo/sv/gem_regs.sv`. If the Demo Testbench is part of a full IP release, the `UVM_REG` model is provided within the IP package (`<GEM_GXL folder>/work/cfg_gen_pkg/fixed_cfgs/reg_models`). For more details, please refer to the GEM_GXL IP documentation [3][4].

2.2. UVM Test Layer

The UVM test layer uses UVM class libraries and associated methodology [2], except that constrained random verification is not used but instead a directed testing approach is used to ensure the demo tests/cases work as required. A hierarchical approach to test sequences is used to enable clarity within test cases.

- All SV tests are located in the `cdn_gem_demo/sve/tests` sub-directory. Descriptions of the tests and all UVM-SV elements are available in HTML via NaturalDocs. For more information, refer to Section 4.
- All sequences libraries are located in the `cdn_gem_demo/sv` subdirectory.

2.3. C Test Layer

The C test layer is provided to promote reuse in a CPU driven system level environment. The C test layer utilizes the same Cadence Core Driver (aka Bare Metal Driver) which is provided with the full IP product for SW development. The core driver provides a simple and consistent API for test cases. It also provides a Hardware Abstraction Layer (aka CPS) which may be adapted to suit different system-level platforms.

- All C based test are located in the `cdn_gem_demo/sve/tests_c` subdirectory. Descriptions of the tests are available in HTML via Doxygen. For more information, refer to Section 4.
- C register headers are included and used within the C test cases. These headers are generated from IPXACT and may be found in the `cdn_gem_demo/c/CoreDriver/include` directory.
- The Core Driver (aka Bare Metal Driver) is normally provided as part of a full IP release, or as a separate SW-only delivery. When it is part of a full IP release, this driver is automatically found by the Demo Testbench Makefile, however it may be overridden via the `CDN_DEMO_SW_PATH_OVERRIDE` Makefile command line option (not required to be set in full IP release).

Figure 2 shows a typical example of the C Test Layers, connected through the C-DPI calls to the Cadence VIP which is wrapped as a UVC. For more details please refer to [6].

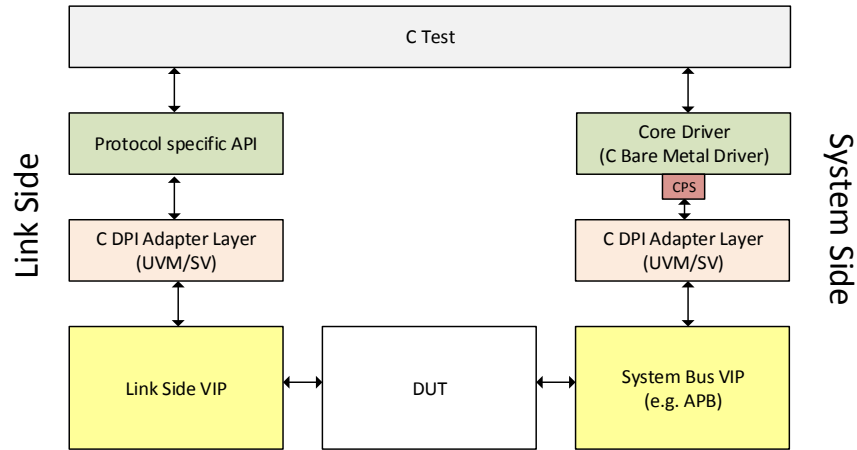


Figure 2: C Test Example Structure

2.4. Cadence GEM_GXL IP

The Demo Testbench architecture for the Cadence GEM_GXL IP is shown in Figure 3. This is encapsulated as a UVC called `cdn_gem_demo` that extends where convenient from a generic UVC called `cdn_demo`. The latter is used as a template across different IP products.

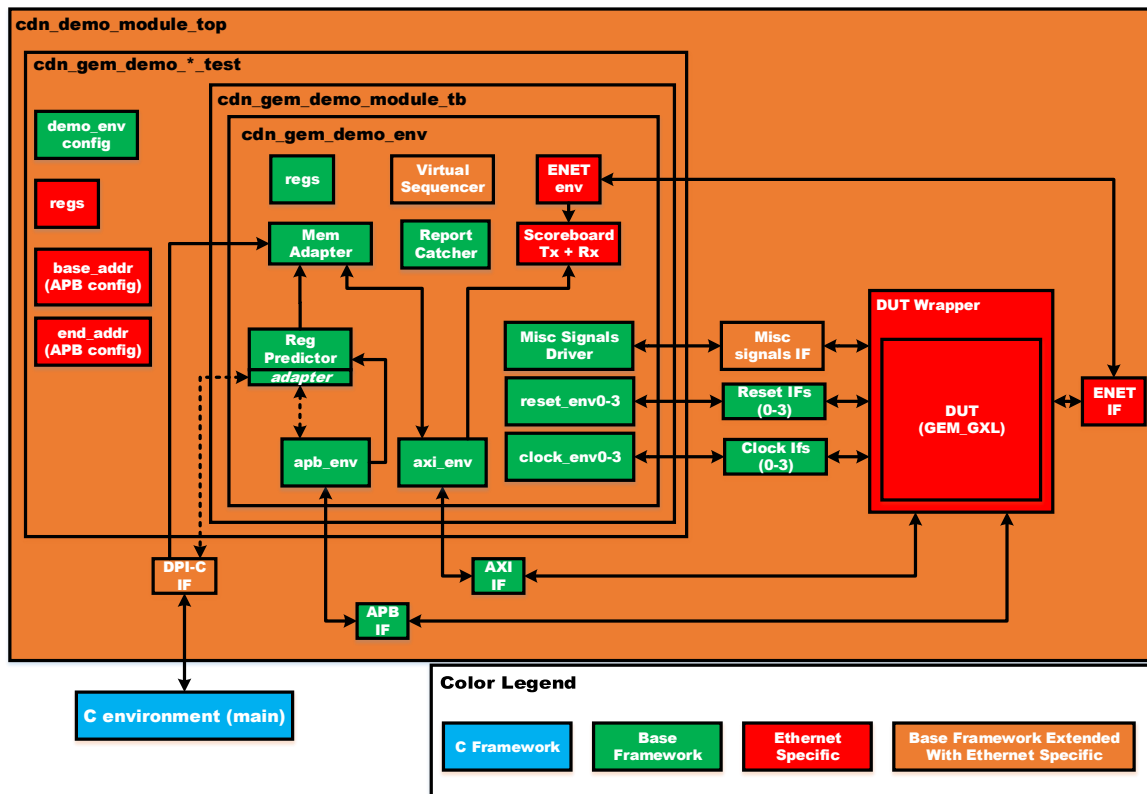


Figure 3: The GEM_GXL Demo Testbench structure.

The following list summarizes the high-level functionalities of the Demo Testbench as contained in the various blocks:

- Clock and reset signals to the DUT/testbench are generated through respective UVCs.

- The default system busses for programming and data structures are APB and AXI, respectively. To allow for an easy replacement with proprietary busses, adapter modules are provided.
- Test sequences run on a virtual sequencer to allow control of the various interface in one place.
- For the UVM test layer, tests follow the standard methodology and are provided to demonstrate basic functionalities.
 - The system memory is emulated within the AXI VIP and handled using its interface.
 - Separate data scoreboards are provided for transmission (Tx) and receive (Rx) paths of the IP. These are configurable from env/sequences and thought to be simple in nature, performing a byte-to-byte comparison between the Ethernet and AXI packets raw data.
- For the C test layer, each test is implemented as a `main` function which is called through the base sequence from the UVM world. Only one C test is compiled for each run.
 - System memory is allocated in the tests and handled through the memory adapter and the C-DPI interface from the underlying UVM environment. To this extent, AXI VIP callbacks are used to modify the content of the AXI transaction with that of the actual C memory.
 - C tests do not rely on the UVM data scoreboard and self-checking mechanisms are directly implemented within each of them in order to reuse this feature at system level.
- For a list of available tests with brief description please refer to section 4. For more detailed information about UVM and C code please refer to documentation as highlighted in sections 4.1 and 4.2.
- For each of the main physical interface in the DUT – i.e. AXI, APB and ENET – a respective SystemVerilog interface is provided. Signals not present in one of the previous are allocated into a miscellaneous signals interface. All interfaces are connected to the DUT through a DUT wrapper.

Support for interrupt monitoring is added in the C/UVM framework. Nevertheless, its operation is not demonstrated in the current GEM_GXL Demo Testbench release.

Please note that this user guide only provides a description of the Demo Testbench in relation to the GEM_GXL IP, which is an advanced and configurable Ethernet controller [1]. It is intended that a full description of the GEM_GXL features is given elsewhere and to this extent reference should be made to the proper documentation [3][4].

3. Package and Directory Structure

This chapter describes how to customize and use the GEM_GXL Demo Testbench.

3.1. Installation

Typically, the Demo Testbench is shipped as either part of a full IP release or standalone as a single file in TAR/GZIP format.

- If provided as a full IP release, please follow the installation instructions provided with the release [3][4].
- If provided as a standalone release, please follow the description below on how to install it to your system.
- If you have requested shipment on other media, follow the instructions on the media label.

To install the project, execute the following steps:

Table 2: Required commands to install the GEM_GXL Demo Testbench.

Command	Description
<code>cp cdnsgemdtb.tar.gz</code>	Copy the CDNSGEMDTB delivery pack file to your local directory
<code>tar -zxvf cdnsgemdtb.tar.gz</code>	Un-tar the archive. As a result, you will have the Demo Testbench directory that contains all deliverables

Please note that the Demo Testbench requires the Cadence VIPCAT to be installed on the customer systems and has been tested with only a subset of simulators; for full details please see section 1.1.

When provided as a standalone package, the Demo Testbench `Makefile` must be told where to find RTL, Software and PHY sources. This is done by setting the following optional `Makefile` command line options:

Table 3: Standalone Package Required Environment Variables.

Variable	Description
<code>CDN_DEMO_TB_PATH_OVERRIDE</code>	Point to a different location for the <code>demo_tb</code> directory. The assumption is that this <code>demo_tb</code> will be a full copy of the directory structure as shown in the "Directory Structure" section of this document.
<code>CDN_DEMO_SW_PATH_OVERRIDE</code>	Point to the "software" container directory
<code>CDN_DEMO_RTL_PATH_OVERRIDE</code>	Point to the "hdl/hdl_src" container directory
<code>CDN_DEMO_PHY_PATH_OVERRIDE</code>	Point to the "phy" container directory

No `Makefile` command line options are needed when the Demo Testbench is provided as part of a full IP release. The Demo Testbench `Makefile` derives all required variables in this case. These `*PATH_OVERRIDE` may also be used within the customer environment to allow the corresponding source to be located as per customer build flow requirements.

3.2. Directory Structure

The directory structure of the GEM_GXL Demo Testbench is shown in Table 4.

Table 4: The directory structure of the GEM_GXL Demo Testbench.

demo_tb	
├── doc	
│ ├── Cadence GEM_GXL Demo TB	
│ └── User Guide.pdf	
├── verif	
│ └── uvc_lib	
│ ├── cdn_clock	
│ ├── cdn_reset	
│ ├── cdn_apb_vip	
│ ├── cdn_axi_vip	
│ ├── cdn_enet_vip	
│ ├── cdn_demo	
│ │ ├── c	
│ │ ├── docs	
│ │ ├── sv	
│ │ ├── sve	
│ │ │ ├── cdn_demo_module_top.sv	
│ │ │ └── Makefile	
│ │ └── vm	
│ ├── cdn_demo_ram_integration	
│ ├── customer_dut	
│ ├── cdn_gem_demo	
│ │ ├── c	
│ │ ├── docs	
│ │ │ ├── docs_doxygen	
│ │ │ └── docs_nd	
│ │ ├── sv	
│ │ ├── sve	
│ │ │ ├── cdn_protocol_demo_module_top.sv	
│ │ │ ├── config.mk	
│ │ │ ├── tests	
│ │ │ └── tests_c	
│ │ └── vm	
│ │ └── *.vsif	

To run a simulation in the GEM_GXL Demo Testbench the end user is intended to use the Demo Testbench Makefile. Its features are described in sections 3.3 and 3.5.

The <DEMO_TB_INSTALL_PATH>/doc folder contains this user guide. The HTML documentation for the code is located at cdn_gem_demo/docs. Both Doxygen (C code) and NaturalDocs (SystemVerilog code) subfolders are present – docs_doxygen and docs_nd respectively.

The GEM_GXL IP instance can be found inside the cdn_gem_demo/sve directory in the protocol-specific top module, i.e. cdn_protocol_demo_module_top.sv. Please refer to this file to have an example of how the GEM_GXL IP can be wrapped, instanced and connected to the testbench.

3.3. Running a Simulation on a Standalone Package

Please note that in order to run the GEM_GXL demo TB a *UVM_REG* SystemVerilog model is needed. This is provided within the GEM_GXL IP package (full IP delivery) or demo TB package (standalone delivery) for the default design configuration. If the end user desires to run the demo TB on a custom configuration should contact Cadence to provide the correct *UVM_REG* model.

This section explains how to run a simulation within the GEM_GXL Demo Testbench using a flexible *Makefile*, which is the recommended approach for a standalone package delivery. For a full IP package please refer to the release notes. The *Makefile* is located inside the *cdn_demo* UVC at *cdn_demo/sve*.

NOTE: it is assumed that the Cadence Incisive and Cadence VIPCAT have all been installed and set up with all relevant environment variables for these tools configured in your current shell. As a minimum, this requires that:

- The *\$DENALI* environment variable is set to point to root directory of the required 64-bit version of the VIPCAT installation (see section 1.1).
- The “*which irun*” command returns valid path to the required version of the Cadence Incisive tool installation (see section 1.1).

The command below need to be executed before running the first simulation. It assures the correct installation of the VIP libraries in the running directory.

```
make -f <DEMO_TB_INSTALL_PATH>/verif/uvc_lib/cdn_demo/sve/Makefile scrub vip
```

The actual simulation (interactive) is run with the following command:

```
make -f <DEMO_TB_INSTALL_PATH>/verif/uvc_lib/cdn_demo/sve/Makefile run_i TEST=<testname>
```

To run a batch mode simulation, please use the “*run*” target instead of the “*run_i*” target:

```
make -f <DEMO_TB_INSTALL_PATH>/verif/uvc_lib/cdn_demo/sve/Makefile run TEST=<testname>
```

For other simulations options, please see the built-in help options. This can be done with the following command:

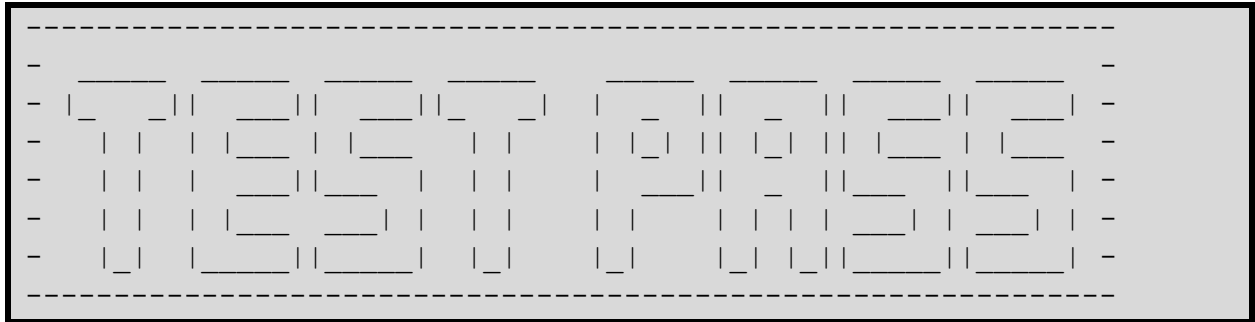
```
make -f <DEMO_TB_INSTALL_PATH>/verif/uvc_lib/cdn_demo/sve/Makefile help
```

Each test will be logged to a separate file as *irun_<testname>.log*. For available *<testname>* values please refer to section 4. Note that calling the *Makefile* without specifying any target and/or arguments will trigger a full (local) regression of available tests. For more information on regressions, please refer to section 3.5.

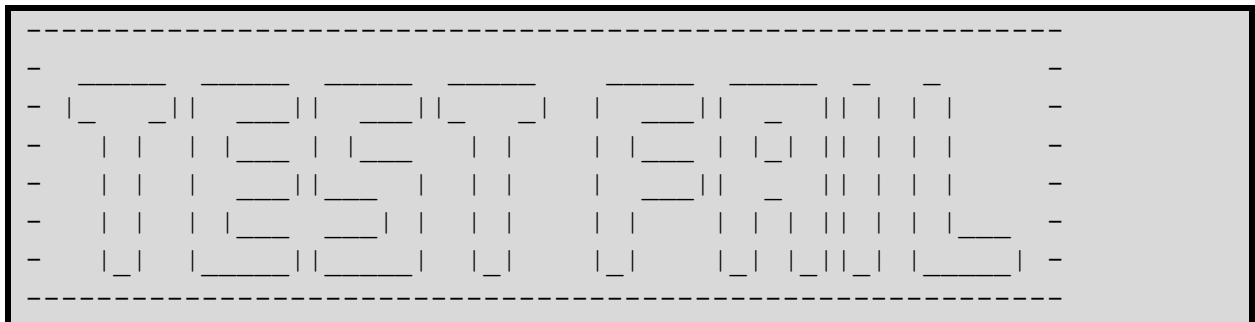
3.4. Test Completion Status – Pass/Fail

The Demo Testbench makes use of the UVM library and as such, reports errors/warnings and fatal errors using the UVM report mechanism. Furthermore, the Cadence VIP is configured to report Denali error message types as UVM errors. Therefore, it is possible to determine the pass/fail status of the test by looking at the UVM end of test report; however, often this can be hard to read and as such, the Demo Testbench reports a pass/fail status as a text banner at the end of the test (during the `uvm_check` phase). The banners are printed as a single `uvm_info` message as shown below:

Message for test pass:



Message for test fail:



Please note: it is possible that other error types (for example assertion errors or tool specific errors) are not reported as `uvm_error` or `uvm_fatal` types. In return, a “grep” of the log file for such errors is advised. Under such conditions the test completion status banner may be incorrect!

3.5. Running Regressions

Please note that in order to run GEM_GXL demo TB regressions a **UVM_REG** SystemVerilog model is needed. This is provided within the GEM_GXL IP package (full IP delivery) or demo TB package (standalone delivery) for a default design configuration. If the end user desires to run the demo TB on a custom configuration should contact Cadence to provide the correct **UVM_REG** model.

Cadence recommends running regressions using eManager/vManager as all applicable test lists are stored in `vsif` format with support for third-party simulators. Furthermore, eManager/vManager is capable of checking the log files for many different source of error message or warning (for example, assertions or UVM errors or Verilog calls to `$error` etc.). Section 3.5.2 details how to use eManager/vManager for regressions.

The GEM_GXL Demo Testbench is also capable of running local regressions in a standalone fashion using the provided `Makefile`. For details on how to run standalone regressions please refer to section 3.5.3.

Common to both approaches are the types of regression. This is documented here in Section 3.5.1.

3.5.1. Types of Regression

For a full list of tests that are included across each of these regression types, please see Section 4. Currently, the following types of regression available depending on the configuration of the design:

- `cdn_gem_demo_regression_uvm_reg`: this regression contains **UVM_REG** tests.
- `cdn_gem_demo_regression_uvm`: this regression contains all of the UVM tests.
- `cdn_gem_demo_regression_c`: this regression contains all of the C tests.
- `cdn_gem_demo_regression_global`: this regression is global and groups UVM and C regressions.

A full description of the regression types can be found by looking at the corresponding eManager/vManager `*.vsif`.

3.5.2. Using eManager/vManager for regressions

Currently, several `.vsif` files are provided in the GEM_GXL Demo Testbench to run regressions using vManager. These are reported below and are available in the `cdn_gem_demo/vm` folder. For a full list of tests that are included across each of these regression types please see section 4.

It is possible to run all of the tests in a regression using Cadence eManager/vManager tool. The files needed for this are located here:

```
<DEMO_TB_INSTALL_PATH>/verif/uvc_lib/cdn_gem_demo/vm
```

The `*.vsif` that control regressions are named using the regression types (as described in Section 3.5.1).

The command below need to be executed before running eManager. It assures starting simulations from a clean working area.

```
make -f <DEMO_TB_INSTALL_PATH>/verif/uvc_lib/cdn_demo/sve/Makefile scrub
```

To run a regression, simply start eManager using the `"start_vm"` target of the Demo Testbench `Makefile`. i.e.:

```
make -f <DEMO_TB_INSTALL_PATH>/verif/uvic_lib/cdn_demo/sve/Makefile start_vm
```

Then from eManager, load the required *.vsif as defined above.

Please note that the `start_vm` target in the Makefile and associated *.vsif assume that you are using LSF for your regression farm and may require editing to work on your local systems. You will notice that `start_vm` target requires an environment variable called `$FARM_VSIF` to be set and pointed to a vsif control file; such a file will need to be created to suit your regression software and farm. Alternatively, you can edit the example shown below. The following file shows a configuration for LSF:

```
//-----
// FILE NAME = customer_farm.vsif
// Extension for LSF parameters
// This file should be pointed to be the environment variable
// $FARM_VSIF.
// This is site dependent and should be set as part of site setup.
// start_vm Makefile target will source this file as a default.
// NOTE: For debugging the DRM, commenting out all the lines
// in this file results in the default 'serial local' scheme.
//-----

drm : <text>lsf</text>;
default_dispatch_parameters : <text>-P $ENV(LSF_PROJECT_STRING) -q
lnx64 -R "OSBIT==64 && OSNAME==Linux && OSREL==EE50 rusage[mem=4000]"
-F 10000000 -W 6:00 -o /dev/null</text>;
```

3.5.3. Running Standalone Regressions

The eManager/vManager regressions are also supported as standalone and can be run directly using the Demo Testbench Makefile. Commands for running standalone regression are reported below.

To run a UVM_REG regression:

```
make -f <DEMO_TB_INSTALL_PATH>/verif/uvic_lib/cdn_demo/sve/Makefile regression_uvm_reg
```

To run a UVM regression:

```
make -f <DEMO_TB_INSTALL_PATH>/verif/uvic_lib/cdn_demo/sve/Makefile regression_uvm
```

To run a C regression:

```
make -f <DEMO_TB_INSTALL_PATH>/verif/uvic_lib/cdn_demo/sve/Makefile regression_c
```

To run a global regression:

```
make -f <DEMO_TB_INSTALL_PATH>/verif/uvic_lib/cdn_demo/sve/Makefile regression
```

Standalone regressions rely on test lists as extracted from the *.vsif files. For each test, a separate log file (`irun_<testname>.log`) is created in the working directory. A summary file reporting the number of warnings/errors is also created as `<regression name>.results`.

3.6. RAM Integration Tests

Please note that this feature is not yet available in the GEM_GXL Demo Testbench and will be added in future releases. This will comprise two methods: i) test cases to exercise the RAM interfaces; ii) swap part of the DUT for a “RAM Integration Stub” that fully exercises all RAM interfaces.

The Cadence IP is provided with generic models for RAMs which are typically replaced by a customer's own technology specific libraries. Please see the full IP documentation for details on RAMs and replacement. After such replacement the customer should verify correct integration of the RAMs.

To assist the customer in this verification, a suite of RAM integration tests is provided as part of the Demo Testbench. These tests are run as per any other test using the `Makefile` or regression scripts, with no additional command line options required and with the same pass/fail status reporting.

These tests aim to prove both connectivity and functionality checks for RAMs which are directly attached to the IP, whether this is within the IP integration layers or within the customer's own design.

3.6.1. Current RAM Integration Limitations

There will be some limitations to RAM integration testing using the Demo Testbench. Due to the fact that the Demo Testbench tests and sequences are supposed to demo the basic working operation of the design IP, all traffic conditions that would exhaustively cover all RAM conditions are not possible to hit. Adding such complexity to the Demo Testbench would significantly increase the complexity of the testbench and as such make it very difficult to use as a vehicle for customer understanding of the programming model required to operate the design. This overall limitation will remain.

4. Use Cases/Demo Cases

A number of tests are provided in the GEM_GXL Demo Testbench to show DUT operations. A list of them can be found Table 5 (C) and Table 6 (UVM).

Table 5: Available C tests in the GEM_GXL Demo Testbench.

Testname	Description
cdn_gem_demo_c_int_enet_txrx_1pkt_test	Loopback test w/ core driver. Transmit and receive one packet.
cdn_gem_demo_c_uc_enet_txrx_1pkt_test	Loopback test w/o core driver. Transmit and receive one packet.
cdn_gem_demo_c_uc_enet_txrx_3pkts_test	Loopback test w/o core driver. Transmit and receive tgree packets.
cdn_gem_demo_c_uc_enet_txrx_q_fixed_scr1_test	Loopback test w/o core driver. Transmit one IPoE packet for each configured queue using the default scheduling algorithm (fixed priority) and receive all of them using screener type 1 filtering.

Table 6: Available UVM tests in the GEM_GXL Demo Testbench.

Testname	Description
cdn_gem_demo_uc_enet_rx_1pkt_test	Demonstrate the basic receive of one Ethernet packet.
cdn_gem_demo_uc_enet_rx_3pkts_test	Demonstrate the basic receive of three consecutive Ethernet packets.
cdn_gem_demo_uc_enet_tx_1pkt_test	Demonstrate the basic transmit of one Ethernet packet.
cdn_gem_demo_uc_enet_tx_2pkts_test	Demonstrate the basic transmit of two consecutive Ethernet packets.
cdn_gem_demo_uc_enet_tx_q_fixed_test	Transmit one packet for each configured queue using the default scheduling algorithm (fixed priority).
cdn_gem_demo_uvm_reg_reset_test	Run the UVM_REG “reset” built in sequence, which reads all the registers and check their reset value.
cdn_gem_demo_uvm_reg_bit_bash_test	Run the UVM_REG “bit-bash” built in sequence, which sequentially walks 0s and 1s through the registers and checks set/clear based on the field access policy.

<code>cdn_gem_demo_uvm_reg_aliasing_test</code>	Run the Cadence UVM_REG “aliasing” built in sequence, which writes a random value to a register and reads all the others to make sure the write did not affect them.
<code>cdn_gem_demo_uvm_reg_read_all_test</code>	Run the Cadence UVM_REG “read” all built in sequence, which reads all the registers in the block.
<code>cdn_gem_demo_uvm_reg_write_all_test</code>	Run the Cadence UVM_REG “write all” built in sequence, which writes all the registers in the block with a random value.
<code>cdn_gem_demo_uvm_reg_write_follow_read_test</code>	Run the Cadence UVM_REG “write follow read” in sequence, which subsequently writes and reads a register.

A full description of available UVM and C tests within the GEM_GXL Demo Testbench can be found in the HTML code documentation contained within the package. This is located at `<DEMO_TB_INSTALL_PATH>/verif/uvc_lib/cdn_gem_demo/docs`. To ensure consistency and test lists that are always up to date, UVM tests are documented using NaturalDocs [7] while C tests are documented using Doxygen [8]. The following sections provide more specific details for NaturalDocs and Doxygen, respectively.

4.1. NaturalDocs UVM-SV VE and Demo Test Documentation

The HTML index for the underlying verification environment and tests can then be found in `<DEMO_TB_INSTALL_PATH>/verif/uvc_lib/cdn_gem_demo/docs/docs_nd/index.html`. Once the index is opened the documentation should be easy to navigate using the menu frame on the left of the browser window. An example can be seen in Figure 4. All test cases should be easily found inside the `cdn_gem_demo` UVC as per the Demo Testbench directory structure.

CADENCE GEM_GXL DEMO TESTBENCH

[INDEX](#)

TESTS

- CDN_DEMO
- CDN_GEM_DEMO (UVM) Library
- COMMON
- USE CASES**
 - Basic Rx Test (One Packet)
 - Basic Rx Test (Three Packets)
 - Basic Tx Test (One Packet)
 - Basic Tx Test (Two Packets)
- REGISTERS
- CDN_GEM_DEMO (C WRAPPER)

SEQUENCES

- CDN_APB_VIP
- CDN_AXI_VIP
- CDN_ENET_VIP
- CDN_CLOCK
- CDN_DEMO
- CDN_GEM_DEMO

FILES

Search

cdn_gem_demo_uc_enet_rx_1pkt_test.sv

This file defines the `cdn_gem_demo_uc_enet_rx_1pkt_test` test.

Summary

cdn_gem_demo_uc_enet_rx_1pkt_test.sv

This file defines the `cdn_gem_demo_uc_enet_rx_1pkt_test` test.

cdn_gem_demo_uc_enet_rx_1pkt_test

This is a Rx test. A packet is received and the system waits to assert the Rx complete.

Summary

cdn_gem_demo_uc_enet_rx_1pkt_test

This is a Rx test.

CLASS HIERARCHY

```

graph TD
    uvm_test --> cdn_demo_base_test
    cdn_demo_base_test --> cdn_gem_demo_base_test
    cdn_gem_demo_base_test --> cdn_gem_demo_uc_enet_rx_1pkt_test
  
```

CLASS DECLARATION

```

class cdn_gem_demo_uc_enet_rx_1pkt_test extends
cdn_gem_demo_base_test
  
```

SEQUENCES

<code>write_dtable</code>	The sequence that writes the descriptor table to the main memory.
<code>init_iq</code>	The sequence that initializes the GEM for single queue operations.
<code>start_rx</code>	The sequence that starts Rx operations.
<code>send_eth_pkt</code>	The sequence that sends an Ethernet packet from the line side.
<code>check</code>	The sequence that check the <code>dpram_fill_dbg</code> register to asses the end of test.
<code>new</code>	The class constructor.
<code>build_phase</code>	This UVM phase is used for building the testbench component hierarchy.
<code>run_phase</code>	This UVM phase performs the DUT simulation.

Figure 4: NaturalDocs documentation example.

The UVM class library is currently documented using the NaturalDocs system. Therefore – to help familiarity among verification engineers – this was also chosen for documenting all the UVM-SystemVerilog code inside the Cadence Demo Testbench.

4.2. Doxygen C Demo Test Documentation

The HTML index for the underlying verification environment and tests can then be found in `<DEMO_TB_INSTALL_PATH>/verif/uvc_lib/cdn_gem_demo/docs/docs_doxygen/html/index.html`. Once the index is opened the documentation should be easy to navigate using the menu frame on the left of the browser window. An example can be seen in Figure 5. All test cases should be easily found in the respective test library. Please note that the core driver is not contained within the GEM_GXL Demo Testbench Doxygen documentation and reference shall be made to [5].

Cadence GEM_GXL Demo Testbench

Main Page	Related Pages	Classes	Files
Related Pages			
Here is a list of all related documentation pages:			
cdn_gem_demo_c_uc_enet_rx_1pkt_test cdn_gem_demo_c_uc_enet_rx_3pkts_test cdn_gem_demo_c_uc_enet_rx_q_scr1_test cdn_gem_demo_c_uc_enet_tx_1pkt_test cdn_gem_demo_c_uc_enet_tx_2pkts_test cdn_gem_demo_c_uc_enet_tx_q_fixed_test cdn_gem_demo_c_int_enet_loopback_test			

Cadence GEM_GXL Demo Testbench

Main Page	Related Pages	Classes	Files
Description			
- AXI memory allocation: - Descriptor table area. - Databuffer area. - Descriptor table initialization: - The first descriptor is pointed to the databuffer area. - The second descriptor is marked with the wrap bits set, i.e. last descriptor in the table. - Print the descriptor table content to check for correct initialization. - APB register initialization: - Store the descriptor table base address in GEM_RECEIVE_Q_PTR. - Disable the other ques by writing a 1 in GEM_RECEIVE_Q1_PTR through GEM_RECEIVE_Q15_PTR. - GEM_DPRAM_FILL_DBG is configured to Rx by default. - Rx transaction: - Write the GEM_NETWORK_CONTROL register for the transaction to start. - Send the packet through the send_line_transaction function. - Check the DMA DPRAM fill level to be zero (GEM_DPRAM_FILL_DBG) to assert the Rx complete. - Print the databuffer memory content to check for transaction correctness. - Test ends: - Free the memory area allocated at the beginning.			

Figure 5: Doxygen documentation example.

The Doxygen tool is widely used by many software teams. Therefore – to help familiarity among engineers working with embedded software/system level verification – this was chosen for documenting all the C code inside the Cadence Demo Testbench.

5. Known Issues

As the GEM_GXL Demo Testbench is released with every DIP delivery – also early adopter and initial one, it stands to reason that there may be known issues/limitations within the current environment. These issues can be in the form of VIP warnings/errors or verification environment warnings/errors. All such issues will be documented here in the following sections:

- VIP related issues are discussed in section 5.1.
- Demo testbench related issues are discussed in section 5.2.
- Test or regression related issues are discussed in section 5.3.

5.1. VIP Related Issues

The following VIP-related warnings are known and will be herein documented:

```
// When AXI VIP spec is set to AXI4
reporter [cdnAxiUvmConfig check] __internal__awlockCheck3 failed: awlock should be 1
bits when (spec_ver_AMBA4 || spec_ver_AMBA5)
reporter [cdnAxiUvmConfig check] __internal__arlockCheck3 failed: arlock should be 1
bits when (spec_ver_AMBA4 || spec_ver_AMBA5)

// When AXI VIP spec is set to AXI3
reporter [cdnAxiUvmConfig check] __internal__awlenCheck3 failed: awlen in AXI3 mode
should be up to 4 bits
reporter [cdnAxiUvmConfig check] __internal__arlenCheck3 failed: arlen in AXI3 mode
should be up to 4 bits
```

The above warnings are fired because the AXI interface in the GEM_GXL IP is not configurable, i.e. it complies with both the AXI3 and AXI4 standards simultaneously. This results in AXI VIP configuration warnings, since the awlock, arlock, awlen and arlen signals are of different size between the two spec versions.

```
UVM_WARNING: uvm_test_top.sve.demo_env.axi_env.active_slave.monitor
[CDN_AXI_NONFATAL_WARNING_SLAVE_DATA_ITEM_CREATION_INVALID_INPUT]
WARNING_SLAVE_DATA_ITEM_CREATION_INVALID_INPUT:
ERROR_WREADY_CONTROL: WreadyControl is OSCILLATING and one of the transfer's channel
delay is 0

UVM_WARNING: uvm_test_top.sve.demo_env.axi_env.active_slave.monitor
[DENALI_CDN_AXI_CB_Error] Agent active_slave dropped transaction with transaction id:
<some id>
```

The above warnings are fired when the AXI VIP slave agent could not create a legal VIP transaction from the input and the item is discarded. This may happen in C-based tests. Nevertheless, their functionality is not affected.

5.2. Demo Testbench Related Issues

Currently nothing to report.

5.3. Test or Regression Related Issues

All documented tests should be running and all regression types should also be running without issues. It is worth nothing – since the GEM_GXL is a highly configurable IP – that care must be taken to run the tests with the right configuration parameters. Please refer to the GEM_GXL documentation [3][4] to have a full picture of all the configuration options available in this IP.

Configurations are defined by *.cee files. Those can be given as input to the GEM_GXL IP configuration tool to generate all the needed defines. As an alternative, you can create a *.cee file using the tool GUI, taking care of properly choosing configuration options. For all the tests, the following lines should be present in the *.cee file of the intended configuration:

```
// Use AXI as host interface
gem_axi axi

// Select DMA over a fixed latency streaming FIFO interface
gem_ext_fifo_interface dma

// Forces the external FIFO interface to not be 8bits wide
gem_fifo_8b_if no

// Host side signalling
gem_hprot_value 0010
gem_axi_prot_value 010
```

For the `cdn_gem_demo_c_int_enet_loopback_test` test, the following must also be set:

```
// Include statistic registers in the design
gem_no_stats include_stats
```

Appendix A. Customer Modifications

The Demo Testbench provides some hooks to enable limited customisations of the DUT and associated verification components. These should be used with care and it is the customer's responsibility to verify any such customisations.

A.1 Replacing the System Side Busses with Alternative UVCs

It is possible that customers will want to replace the system side busses with their own proprietary interfaces. Typically, this would be done by adding bridges to the IP RTL on top of AXI and APB as well as adding UVCs to the demo TB. To make this simpler for our customers we have done the following:

1. For the DUT APB Slave interfaces (APB Master VIP), `UVM_REG` is used to abstract register reads/writes, thus only the register adapter file (`cdn_demo_sys_bus_reg_adapter.sv`) needs to be replaced or modified. As a minimum the customer should re-implement the following functions:
 - `read()`: only applicable for C tests to trigger an APB Master VIP read.
 - `write()`: only applicable for C tests to trigger an APB Master VIP write.
 - `reg2bus()`: only applicable for UVM tests to convert `UVM_REG` operation to APB Master transaction.
 - `bus2reg()`: applicable to both UVM and C tests to monitor APB Master transaction.
2. For the DUT AXI Master interface (AXI Slave VIP), a memory model is used to emulate attached system memory and is located within the AXI Slave VIP for UVM tests and within the C test layer when C tests are used. The system memory model must be accessible from both test layer side and VIP side. As such all reads/writes to the system memory model are encapsulated into a single file called `cdn_demo_sys_bus_memory_adapter.sv`. As a minimum the customer should re-implement the following functions:
 - `read_mem()`: only applicable for UVM tests to fetch data from system memory model located within the AXI Slave VIP.
 - `write_mem()`: only applicable for UVM tests to push data into system memory model located within the AXI Slave VIP.
 - `write_cdn_axi_uvm_driver_BeforeSendResponse()`: applicable for C and UVM tests to fetch data from the system memory model and provide to AXI Slave VIP on callback from VIP after AXI address phase. Data returned is applied to the data fields at the AXI data phase.
 - `write_cdn_axi_uvm_driver_Ended()`: applicable for C and UVM tests to push data from AXI Slave VIP into the system memory model upon callback from VIP at end of AXI transaction. AXI write data is written at the end of the transaction.
3. Control knobs are provided to switch the APB and AXI VIP from active to passive or else to disabled completely. Customer should change the following variables in the Demo Testbench config object (`cdn_demo/sv/cdn_demo_config_object.sv`):
 - `has_active_apb` – set to 0 to change APB VIP to passive monitor.
 - `has_active_axi` – set to 0 to change AXI VIP to passive monitor.
 - `has_passive_apb` – optionally set to 0 to disable APB VIP.
 - `has_passive_axi` – optionally set to 0 to disable AXI VIP.

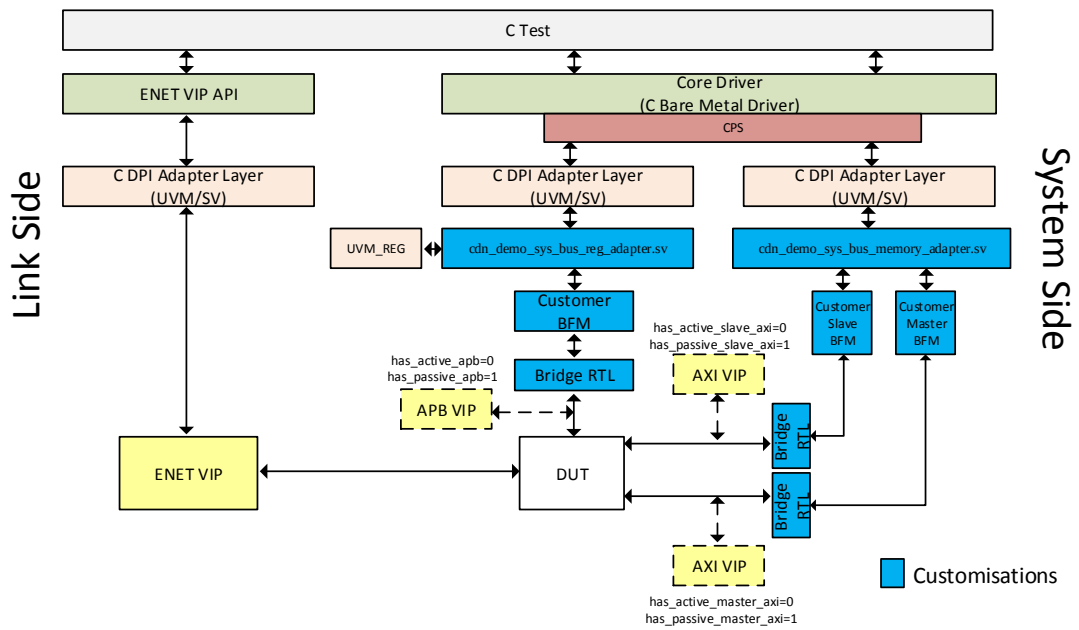


Figure 6: Required Customisations System Bus Replacement.

An example to highlight the areas which require customisations to replace the system bus VIP with customer proprietary BFM is shown in Figure 6. In this example, the original AXI/APB VIP remain as passive monitors to aid debug.

A.1.1 Recommended Tests for Validating System Bus Replacement

For integration of a bus bridge over the APB bus we recommend using the register tests as they run quickly and exercise the APB bus well. For integration of a bus bridge over the AXI bus we recommend using C/UVM one packet transmit tests, since they perform simple accesses the host memory. All of these tests can be found easily in the Doxygen/NaturalDocs HTML documentation described in Section 4.